

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



APPLICATIONS OF NATURAL LANGUAGE PROCESSING IN INDUSTRY

Project 3

BUILD A BANKING AI-AGENTS

Lecturer: Dr. Nguyen Hong Buu Long

Ho Chi Minh City, 04/2026

Contents

1	Project requirements	2
2	Task requirements	3
2.1	System design	3
2.2	Run LLM model	3
2.3	Video demonstration	3
2.4	Source code structure and implementation guidance	4

1 Project requirements

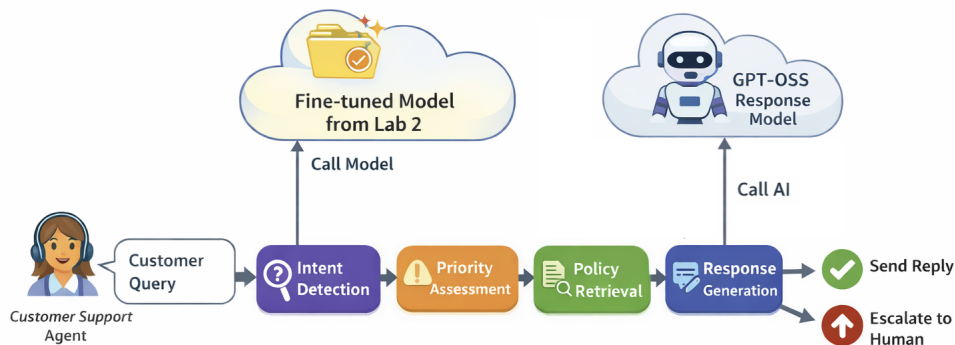


Figure 1: High-level workflow of AI Agent in this project

The objective of this lab is to design and implement a simple **AI agentic workflow** for customer support in the banking domain. The system should be able to receive a customer message, identify the corresponding intent, retrieve relevant policy information, generate a draft response, and determine whether the case can be handled automatically or should be escalated to human staff. Specifically, students are required to complete the following tasks:

- Build an AI agentic pipeline for handling customer issues in the banking domain.
- Implement at least one **intent detection node** as a core component of the workflow.
- Add several additional basic nodes to complete the workflow, such as priority estimation, policy retrieval, response drafting, validation, or escalation.
- Integrate an LLM-based component if needed. If students choose to use an AI model, they are encouraged to use **gpt-oss-20b**, an open-weight model designed for local, low-latency, and specialized use cases. OpenAI states that **gpt-oss-20b** is suitable for local inference and agentic workflows, but it is not served through the OpenAI API.
- Demonstrate the complete workflow on a set of example banking customer requests.

2 Task requirements

2.1 System design

- Students must design an agentic workflow consisting of multiple nodes. The system does not need to be a fully autonomous agent; a structured workflow or a simple state-machine design is sufficient.
- The workflow must include the following mandatory node:
 - **Intent Detection Node:** identifies the customer intent from the input message.
 - **Priority or Risk Detection Node:** determines whether the issue is low, medium, or high priority.
 - **Policy Retrieval Node:** retrieves a relevant FAQ entry, policy snippet, or support guideline.
 - **Response Drafting Node:** generates a draft reply for the customer.
 - **Validation Node:** checks whether the generated response is consistent with the policy and whether important information is missing.
 - **Escalation Node:** decides whether the case should be transferred to a human support agent.

2.2 Run LLM model

- Student can make use of GPUs free of Google Colab to run **gpt-oss-20b** by using Ollama. Afterward, use Pinggy to forward the local port of Ollama (localhost:11434) to a public port (**typical type:** `http://*.a.free.pinggy.link`) that students can call as an API of LLM. **Refer to the given notebook:** *Ollama-Pinggy.ipynb*.
- Students are encouraged to read [Ollama documents](#) for more details.

2.3 Video demonstration

- Students must submit one short video demonstrating the workflow.
- The video should show:
 - One or more example customer messages.

- The outputs of the main workflow nodes.
- The final response or escalation decision.
- The recommended duration is **2–5 minutes**.

2.4 Source code structure and implementation guidance

All source code should be committed to GitHub in order to create favorable conditions for the assessment process.

Students can organize the source code according to the following structure (see Figure. 2):

```
banking-agentic/  
├── README.md  
├── requirements.txt  
├── run.py  
├── app/  
│   ├── main.py  
│   ├── core/  
│   │   ├── settings.py  
│   │   └── schemas.py  
│   ├── data/  
│   │   └── policies.py  
│   ├── clients/  
│   │   ├── base.py  
│   │   └── ollama_client.py  
│   ├── nodes/  
│   │   ├── intent_node.py  
│   │   ├── priority_node.py  
│   │   ├── policy_node.py  
│   │   ├── draft_node.py  
│   │   ├── validation_node.py  
│   │   └── router_node.py  
│   └── agent/  
│       └── orchestrator.py  
└── examples/  
    └── sample_requests.json
```

Figure 2: Sample of source code organization

The purpose of each file is described below.

README.md

- Describe the overall objective of the project.
- Explain the workflow of the banking agentic system.
- Provide instructions for installing dependencies and running the project.
- Provide the URL of the video demo.

requirements.txt

- List all required Python packages.

run.py

- Provide the entry point for running the application.
- This file should start the FastAPI server.
- Students should keep this file simple and use it only to launch the project, not to store business logic.

app/main.py

- Create and configure the FastAPI application.
- Register the API routes and initialize the main server object.

app/core/settings.py

- Store all application settings and environment-based configuration.
- Define the Ollama server URL, the response generation model, and the path or identifier of the fine-tuned intent model from Lab 2.

app/core/schemas.py

- Define the input and output schemas used throughout the system.
- Include request schema, response schema, node outputs, and trace format.
- This file should also contain structured result objects for intent detection, priority classification, policy retrieval, response drafting, validation, and routing.

app/data/policies.py

- Store the policy data or FAQ snippets used by the system.
- In fact, this system will query the database to get FAQs or anything. However, students just need to create some dummy data that simulates real data in this project rather than building a database.

app/clients/base.py

- Define the base client interface for model calling.
- Since this project only uses Ollama, this file can still be used as an abstraction layer so that all model-calling components follow a consistent interface.

app/clients/ollama_client.py

- Implement the Ollama client used to call the language model.
- This file should handle sending prompts to Ollama and receiving model outputs.

app/nodes/intent_node.py

- Implement the intent detection node.
- This node must use the fine-tuned model obtained in Lab 2.
- Its responsibility is to receive a customer message and return the predicted banking intent.
- Students may design this node as a wrapper around the Lab 2 inference class or checkpoint loader.
- This is the core node of the workflow and should be treated as the main specialized classifier in the system.

app/nodes/priority_node.py

- Implement a simple priority or risk detection node.
- This node may use rules, keywords, or lightweight logic to classify whether a case is low, medium, or high priority.
- Example: issues involving suspicious transactions, blocked accounts, or lost money may be marked as high priority.

app/nodes/policy_node.py

- Implement the policy retrieval node.
- Given the predicted intent, this node returns the relevant policy or guideline from `policies.py`.
- The purpose of this node is to ground the final response in predefined support knowledge.

app/nodes/draft_node.py

- Implement the response drafting node.
- This node may call Ollama to generate a draft reply to the customer.
- Students are encouraged to use `gpt-oss` in Ollama for this component.
- The draft should take into account the customer message, the detected intent, the priority level, and the retrieved policy.
- The output may include the draft reply, missing information, and the next suggested action.

app/nodes/validation_node.py

- Implement a validation node for checking whether the generated draft is acceptable.
- This node may verify whether the response is too short, whether important information is missing, or whether the detected intent confidence is too low.
- Students may keep this node simple, but it should help improve reliability and transparency.

app/nodes/router_node.py

- Implement the final decision node of the workflow.
- This node decides whether the system should:
 - Send a reply directly.
 - Ask the customer for more information.
 - Or escalate the case to a human support team.
- The routing decision should be based on the outputs of the previous nodes.

app/agent/orchestrator.py

- Implement the main workflow controller.
- This file is responsible for calling all nodes in the correct order:
 - Intent detection.
 - Priority detection.

- Policy retrieval.
 - Response drafting.
 - Validation.
 - Routing.
- It should also collect intermediate outputs so that the workflow trace can be observed during testing.
 - This file represents the overall AI agentic pipeline.

examples/sample_requests.json

- Provide several example banking customer messages for testing the system.
- The examples should cover multiple intents, such as transfer failure, card not received, blocked account, or refund request.
- Students may use this file to demonstrate the workflow and verify whether the outputs are reasonable.